

Simulador de Energía Eléctrica en una Red de Distribución

1. Descripción del proyecto

El objetivo de esta práctica es desarrollar un simulador de red eléctrica simplificada inspirada en España. La idea principal es reproducir situaciones de fallo o apagón dentro de la simulación, entender por qué se producen y plantear soluciones que reduzcan su impacto o eviten que vuelvan a ocurrir.

La red no pretende copiar con exactitud la infraestructura real, sino construir una versión simplificada que sea lo suficientemente representativa como para que la lógica del sistema sea comprensible y defendible. Los nodos del grafo representan generadores de energía y ciudades consumidoras, y las aristas representan las líneas de transmisión que los conectan.

El programa está organizado en bloques independientes que se fueron desarrollando y probando por separado desde el módulo principal antes de integrarse. Las librerías utilizadas son `networkx` para modelar y operar sobre el grafo, `matplotlib` para visualizar la red y `random` para generar situaciones de fallo variables en cada ejecución.

2. Estructura de la red

La red se representa como un grafo no dirigido con 21 nodos en total: 8 generadores y 13 ciudades. Los generadores tienen una cantidad de energía disponible medida en MW. Las ciudades tienen una demanda mínima que debe ser cubierta para considerarse suministradas.

Las aristas modelan las líneas de transmisión. Cada una almacena la distancia en kilómetros, el tipo de línea (alta tensión o normal), la capacidad máxima que puede transportar y un peso que representa la pérdida energética esperada en ese tramo. Ese peso es el producto de la distancia por un coeficiente que depende del tipo de línea.

Las líneas de alta tensión tienen un coeficiente de 0.00002 y las normales de 0.00008, cuatro veces mayor. Esto significa que una línea de alta tensión puede ser más larga geográficamente y aun así ser preferida por el algoritmo, porque pierde mucha menos energía por kilómetro recorrido.

3. Algoritmo de Dijkstra: funcionamiento e implementación

El núcleo del simulador es el algoritmo de Dijkstra, que resuelve el problema del camino de mínimo coste entre dos nodos en un grafo con pesos no negativos. En este proyecto el coste no es la distancia física sino la pérdida energética acumulada a lo largo de la ruta, lo que hace que el algoritmo elija caminos eficientes aunque no sean los más cortos en kilómetros.

3.1 Cómo funciona

Dijkstra parte del nodo origen con coste cero y mantiene una cola de prioridad con el coste mínimo conocido para llegar a cada nodo. En cada paso extrae el nodo con menor coste acumulado, revisa sus vecinos y actualiza sus costes si encuentra un camino más barato. Repite el proceso hasta haber procesado todos los nodos alcanzables.

En este simulador se llama directamente a `nx.dijkstra_path(G, generador, ciudad, weight='peso')`, que devuelve la lista de nodos que forman la ruta óptima entre un generador y una ciudad usando el atributo peso de cada arista como coste.

3.2 Distribución en dos fases

La distribución de energía no llama a Dijkstra de forma directa para todas las ciudades. Primero recorre cada ciudad y comprueba si tiene algún generador como vecino directo en el grafo. Si lo hay y tiene energía suficiente y la línea no supera su capacidad, se asigna ese generador sin necesidad de ejecutar el algoritmo. Esta primera fase elimina la mayoría de las llamadas a Dijkstra en condiciones normales.

Las ciudades que no quedan cubiertas pasan a la segunda fase, donde sí se aplica Dijkstra. Para cada combinación de generador y ciudad se calcula la ruta de mínima pérdida, se estima la energía que hay que enviar para compensar las pérdidas del trayecto y se comprueba que ninguna arista de la ruta supere su capacidad acumulada.

3.3 Cálculo de pérdidas y energía

La pérdida de una ruta se calcula sumando tramo a tramo:

$$\text{perdida_pct} = \sum (\text{distancia_km} \times \text{coeficiente} \times 100) \quad [\text{máximo } 80\%]$$

Con ese porcentaje se obtiene la energía que debe enviar el generador para que llegue suficiente a la ciudad:

$$\text{energia_a_enviar} = \text{demanda} / (1 - \text{perdida_pct} / 100)$$

Y la energía que efectivamente llega:

$$\text{energia_recibida} = \text{energia_a_enviar} \times (1 - \text{perdida_pct} / 100)$$

Una ciudad se considera suministrada si la energía recibida alcanza al menos el 98% de su demanda mínima. El margen del 2% evita que pequeños errores de redondeo en punto flotante den lugar a clasificaciones incorrectas.

3.4 Comparación con el método subóptimo

Para ilustrar el valor de Dijkstra, el simulador incluye un método alternativo que minimiza kilómetros en lugar de pérdidas, usando `weight= 'distancia_km'` en la llamada al algoritmo. Este método puede seleccionar líneas normales cortas con alta pérdida energética, lo que demuestra que elegir la ruta físicamente más corta no equivale a elegir la más eficiente. La comparación entre ambos métodos se muestra explícitamente en la salida del programa.

4. Simulación de fallos y redistribución

El programa genera aleatoriamente un fallo (60% de probabilidad) o un apagón (40%). Los fallos pueden ser leves, moderados o críticos según el número y la importancia de los recursos afectados. Los apagones tienen cuatro variantes: corte de la línea más crítica de la red, apagón regional, caída de un generador principal o fallo combinado.

En todos los casos, el procedimiento es el mismo. Se hace una copia profunda del grafo original para no modificarlo, se aplican los cambios sobre esa copia (eliminando aristas o reduciendo la energía disponible de un generador) y se calcula el estado de impacto inicial, que refleja qué ciudades pierden el suministro porque su ruta usaba una línea cortada o su generador ha sido afectado.

Para que los incidentes críticos sean realistas, el simulador analiza previamente cuántas rutas activas dependen de cada arista. Cuando hay que seleccionar una línea impactante, se elige entre las tres más usadas de la red en ese momento.

Tras el impacto, el sistema reinicia el estado del grafo dañado y vuelve a ejecutar el algoritmo de distribución completo. Dijkstra busca rutas alternativas usando los generadores y líneas que siguen activos. Las ciudades que aun así no pueden ser cubiertas pasan al módulo de diagnóstico, que identifica la causa del problema y propone una ruta de solución ignorando las restricciones de capacidad para mostrar qué sería posible si se reforzaran las líneas afectadas.

5. Análisis de eficiencia

5.1 Complejidad

Dijkstra implementado con un montículo binario, que es el que usa NetworkX internamente, tiene una complejidad de $O((V + E) \log V)$ por llamada, donde V es el número de nodos y E el número de aristas. Con $V = 21$ y $E \approx 36$, el coste por llamada es prácticamente nulo.

La distribución completa ejecuta Dijkstra para cada combinación posible de generador y ciudad, lo que da una complejidad total de $O(G \times C \times (V + E) \log V)$ con $G = 8$ generadores y $C = 13$ ciudades. Esto equivale a unas 18.000 operaciones elementales por ejecución, una cantidad insignificante en cualquier hardware moderno.

El análisis de uso de aristas y el cálculo del impacto inicial son operaciones lineales en el número de tramos activos y se ejecutan una sola vez por escenario.

5.2 Decisiones de diseño

- Fase 1 de vecinos directos: reduce el número de llamadas a Dijkstra cubriendo con generadores adyacentes sin pasar por el algoritmo.
- Caché del análisis de aristas: el análisis de uso de aristas se ejecuta una sola vez sobre el estado normal y su resultado se reutiliza tanto para fallos como para apagones.
- Copia profunda del grafo: permite simular incidentes sin alterar el grafo original, facilitando la comparación entre el estado normal y el estado tras el fallo.
- Cota de pérdida al 80%: evita valores incoherentes en rutas extremadamente largas sin añadir coste computacional.
- Registro de carga por arista: verificar si una ruta supera la capacidad de sus líneas cuesta $O(\text{longitud de la ruta})$, no $O(E)$.

5.3 Limitaciones del modelo

- El modelo de pérdida es lineal con la distancia. Una red real requeriría ecuaciones de flujo de potencia, como Newton-Raphson o el modelo DC power flow, para ser precisa.
- Para redes de mayor tamaño, la búsqueda exhaustiva por pares generador-ciudad podría sustituirse por un algoritmo SSP inverso desde cada ciudad, reduciendo el número de llamadas a Dijkstra.
- No se modelan restricciones de rampa de generadores, costes de arranque ni retrasos asociados a la conmutación de rutas.